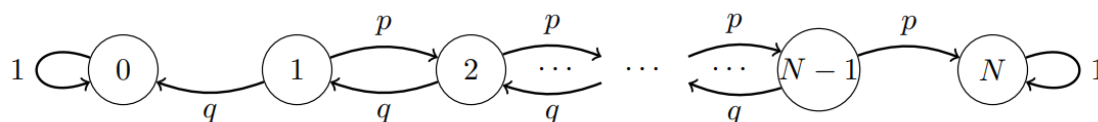


Gambler's Ruin

Navansh Bansal

Introduction

Consider a gambler betting on the outcome of a sequence of independent biased coin tosses. If the coin comes up heads with probability p , she adds one dollar to her purse; if the coin lands tails up with probability $q = 1 - p$, she loses one dollar. If she ever reaches a fortune of N dollars, she will stop playing. If her purse is ever empty, then she must stop betting.



Question 1

Let τ denote the time at which the gambler is either ruined (wealth is 0) or wins the game (wealth is N).

a) What is the probability that the gambler wins the game given that she starts with an initial wealth of k dollars?

Solution:

Gambler starts at an initial wealth of k . Let the probability that the gambler wins i.e reaches final state of N , from this starting state be $T(k)$

Then we can obtain a recursive relation as follows:

$$T(k) = pT(k+1) + qT(k-1)$$

where we have:

- p = Probability of moving to the next state
- q = Probability of moving to the previous state
- $q = 1 - p$
- Base conditions: $T(0) = 0$ and $T(N) = 1$

The characteristic equation for the above recursion is :

$$x = px^2 + q$$

Solving it further we get:

$$x = px^2 + 1 - p$$

$$x - 1 = p \times (x - 1) \times (x + 1)$$

$$(x - 1) \times (px + p - 1) = 0$$

Which has roots $x = 1$ and $\frac{1-p}{p}$ or $\frac{q}{p}$

Making 2 cases on whether the roots are equal or not for this linear homogeneous recursion:

Case 1: $p \neq q$

$$T(k) = A \times (1)^k + B \times \left(\frac{q}{p}\right)^k$$

Case 2: $p = q$

$$T(k) = A \times (1)^k + B \times k(1)^k$$

Where A and B are constants, depending on the base conditions which are $T(0) = 0$ and $T(N) = 1$.

Solving for Case 1:

$$\begin{aligned} 0 &= A + B \\ 1 &= A + B \times \left(\frac{q}{p}\right)^N \end{aligned}$$

Which upon solving give:

$$\begin{aligned} A &= \frac{1}{1 - \left(\frac{q}{p}\right)^N} \\ B &= \frac{-1}{1 - \left(\frac{q}{p}\right)^N} \end{aligned}$$

Putting in the values of the constants in the recursive equation,
The probability that the gambler wins the game given she starts with an initial wealth of k dollars is:

$$T(k) = \frac{1 - \left(\frac{q}{p}\right)^k}{1 - \left(\frac{q}{p}\right)^N}$$

Solving for Case 2:

$$\begin{aligned} 0 &= A \\ 1 &= A + BN \end{aligned}$$

Which upon solving give:

$$\begin{aligned} A &= 0 \\ B &= \frac{1}{N} \end{aligned}$$

Putting in the values of the constants in the recursive equation,
The probability that the gambler wins the game given she starts with an initial wealth of k dollars is:

$$T(k) = \frac{k}{N}$$

b) If we let $N \rightarrow \infty$ and let the player gamble forever unless she is ruined, what is the probability that she amasses an infinite amount of wealth?

Solution:

Case 1: $p \neq q$

$$\lim_{N \rightarrow \infty} T(k) = \lim_{N \rightarrow \infty} \frac{1 - \left(\frac{q}{p}\right)^k}{1 - \left(\frac{q}{p}\right)^N}$$

Case 2: $p = q$

$$\lim_{N \rightarrow \infty} T(k) = \lim_{N \rightarrow \infty} \frac{k}{N}$$

$$\lim_{N \rightarrow \infty} T(k) = \begin{cases} 0 & \text{if } q \geq p \text{ or } p \leq \frac{1}{2} \\ 1 - \left(\frac{q}{p}\right)^k & \text{if } p > \frac{1}{2} \end{cases}$$

c) What is the expected value of the number of rounds the gambler plays until she is ruined or wins the game when she starts from an initial wealth of k dollars? ($0 \leq k \leq N$)

Solution:

Let $\mathbb{E}_i$ denote the expected number of rounds till game ends if initial wealth is i . We can represent this situation using a recurrence relation as follows:

$$\mathbb{E}_i = p \times \mathbb{E}_{i+1} + q \times \mathbb{E}_{i-1} + 1$$

Base Cases: $\mathbb{E}_0 = \mathbb{E}_N = 0$

The solution to this linear non-Homogeneous recurrence can be divided into two parts: Homogeneous and Particular solutions.

The Homogeneous part has roots: 1 and $\frac{q}{p}$

Case 1: $p \neq q$

- Homogeneous part has roots 1 and $\frac{q}{p}$
- The particular solution is $\mathbb{E}_i = i(a_0)$

Reason: As the Homogeneous part of the equation also has root 1, so in case of such repetition one root is 1 and other i .

Solving for the constant a_0 :

$$ia_0 = p(i+1)a_0 + q(i-1)a_0 + 1 \implies 1 = a_0(q-p)$$

Total value of $\mathbb{E}_i = \mathbb{E}_i^{\{p\}} + \mathbb{E}_i^{\{h\}}$

$$\mathbb{E}_i = \frac{i}{q-p} + A \times (1)^i + B \times \left(\frac{q}{p}\right)^i$$

Where A and B are constants depending on the base cases:

$$A + B = 0$$

$$\frac{N}{q-p} + A + B \times \left(\frac{q}{p}\right)^N = 0$$

So, we get:

$$A = \frac{\frac{N}{p-q}}{1 - \left(\frac{q}{p}\right)^N}$$

$$B = \frac{\frac{N}{q-p}}{1 - \left(\frac{q}{p}\right)^N}$$

$$\mathbb{E}_i = \frac{i}{q-p} + \frac{N}{p-q} \times \frac{1 - \left(\frac{q}{p}\right)^i}{1 - \left(\frac{q}{p}\right)^N}$$

Case 2: $p = q$

- Homogeneous part has roots 1 and i
- The particular solution is $\mathbb{E}_i = i^2(a_0)$

Reason: As the Homogeneous part of the equation already has repeated root of 1, so in case of such repetition one root is 1 and other i , while for particular solution it would become $i^2(a_0)$.

Solving for the constant a_0 :

$$i^2 a_0 = p(i+1)^2 a_0 + q(i-1)^2 a_0 + 1 \implies 1 + a_0 = 2ia_0(q-p) = 0$$

Total value of $\mathbb{E}_i = \mathbb{E}_i^{\{p\}} + \mathbb{E}_i^{\{h\}}$

$$\mathbb{E}_i = -i^2 + A \times (1)^i + B \times i(1)^i$$

Where A and B are constants depending on the base cases:

$$A = 0$$

$$-N^2 + A + BN = 0$$

So, we get:

$$A = 0$$

$$B = N$$

$$\mathbb{E}_i = -i^2 + iN$$

Question 2

The gambler is tired of placing \$1 bets and decides to play aggressively. If she currently has a wealth of $k < \frac{N}{2}$, she bets the entire amount, and either gets ruined with probability q or doubles her wealth with probability p . If she has $k \geq \frac{N}{2}$, she only bets the difference $N - k$. She will win the game with wealth N with probability p , or she will lose her bet of $N - k$ with probability q and continues betting.

Note : For this problem, you may assume that the ratio of the initial wealth to the maximum wealth $\frac{k}{N}$ has a finite binary decimal expansion. i.e., $\frac{k}{N} = \sum \left(\frac{1}{2}\right)^{n_k}$ where the finite increasing sequence $\{n_k\}$ represents the positions of the 1's in the binary representation of $\frac{k}{N}$

a) What is the probability of the gambler winning the betting game when she starts with an initial wealth of k ? ($0 \leq k \leq N$)

Solution:

The idea is to represent $\frac{k}{N}$ in binary form, i.e., $.n_1n_2\cdots n_m$, where $0 \leq n_i \leq 1$ and m is finite.

$$\frac{k}{N} = \sum n_i \left(\frac{1}{2}\right)^i$$

For $\frac{k}{N} \geq \frac{1}{2}$, we must have $n_1 \neq 0$ or $n_1 = 1$.

- If $0 < k < \frac{N}{2}$, the gambler continues the game and with p probability shifts the bits of $\frac{k}{N}$ left by 1 or with q probability all n'_i s go 0 and game ends, this continues till $n_0 \neq 1$ or all n'_i s go 0
- If $\frac{N}{2} \leq k < N$, the gambler continues the game and with p probability finishes the game with N wealth or with q probability ends up at $2k - N$, which comes back to first case or ends the game with 0 wealth of $k = \frac{N}{2}$.

This $2k - N$ can be cleverly represented in the binary bit-manipulation of $\frac{k}{N}$ by shifting all bits to left by 1 and then dropping the first number from left

$$P(x) = \begin{cases} 0 & x = 0 \\ pP(2x) & 0 < x < \frac{N}{2} \\ qP(2x - N) + p & \frac{N}{2} \leq x < N \\ 1 & x = N \end{cases}$$

To solve this recursion in the binary bit-manipulation way we can simply traverse the binary representation of $\frac{k}{N}$:

- If the i^{th} bit is '0' then go to the $i + 1^{\text{th}}$ bit while multiply the probability with p for 1 win, this is equivalent to shifting all bits left by 1 or $P(2x)$ and then multiplying by p i.e $pP(2x)$.
- If the i^{th} bit is '1' then go to the $i + 1^{\text{th}}$ bit while multiply the probability with q for 1 lose and add p probability for single/complete win, this is equivalent to shifting all bits left by 1 and dropping one bit from left or $P(2x - N)$ and then multiplying by q i.e $qP(2x - N)$ and then adding p i.e $qP(2x - N) + p$.

b) What is the expected duration of the above game? (The game ends when the gambler is ruined or wins the game)

Solution: Let $\mathbb{E}(x)$ represent the expected number of rounds for game to end if instantaneous wealth is x .

For the first case where $0 < k < \frac{N}{2}$, we can obtain the recursive equation :

$$\mathbb{E}(x) = p\mathbb{E}(2x) + p + q\mathbb{E}(0) + q$$

$$\mathbb{E}(x) = 1 + p\mathbb{E}(2x)$$

For the second case where $\frac{N}{2} \leq x < N$, we can obtain a recursion equation :

$$\mathbb{E}(x) = q\mathbb{E}(2x - N) + q + p$$

$$\mathbb{E}(x) = q\mathbb{E}(2x - N) + 1$$

$$\mathbb{E}(x) = \begin{cases} 0 & x = 0 \\ 1 + p\mathbb{E}(2x) & 0 < x < \frac{N}{2} \\ 1 + q\mathbb{E}(2x - N) & \frac{N}{2} \leq x < N \\ 0 & x = N \end{cases}$$

To solve this recursion in the binary bit-manipulation way we can simply traverse the binary representation of $\frac{k}{N}$:

- If the i^{th} bit is '0' then go to the $i + 1^{\text{th}}$ bit while multiply the expectation with p for 1 win, this is equivalent to shifting all bits left by 1 or $\mathbb{E}(2x)$ and then multiplying by p i.e $p\mathbb{E}(2x)$ and adding 1 i.e $1 + p\mathbb{E}(2x)$.
- If the i^{th} bit is '1' then go to the $i + 1^{\text{th}}$ bit while multiply the expectation with q for 1 lose and add p probability for single/complete win, this is equivalent to shifting all bits left by 1 and dropping one bit from left or $\mathbb{E}(2x - N)$ and then multiplying by q i.e $q\mathbb{E}(2x - N)$ and then adding 1 i.e $1 + q\mathbb{E}(2x - N)$.

Question 3

The gambler is having a streak of bad luck. She starts with an initial fortune of k and wins or loses each bet with probability p and q , respectively. But her curse is that if she ever has a wealth of m , she is not able to attain a wealth higher than $m + W$. That is, after attaining a wealth of m , if she manages to amass her wealth to $m + W$, she will always lose in the next round. Noticing this, she decides to count her losses and stop playing when her wealth reaches t . How many rounds is she expected to gamble for? ($0 < t < k < N - W$)

Note : In this part, it can be shown that the answer can be expressed as an irreducible fraction $\frac{a}{b}$, where p and q are integers and $b \not\equiv 0 \pmod{M}$. Output the integer equal to $a \times b^{-1} \pmod{M}$. In other words, output such an integer x that $0 \leq x < M$ and $x \times b \equiv a \pmod{M}$. Let $M = 10^9 + 7$.

Solution:

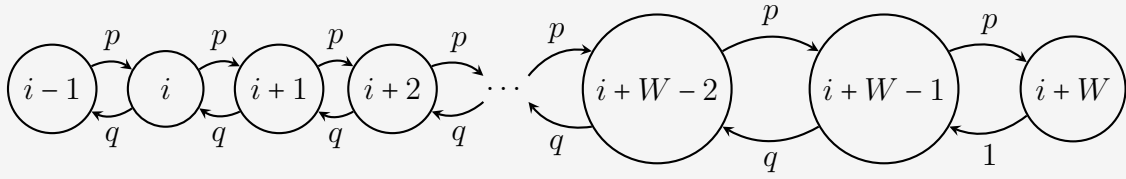
Let $\mathbb{E}_i$ denote the expected number of rounds till game ends if initial wealth is i . We can represent this situation using a recurrence relation as follows:
(Linearity of Expectation)

$$\mathbb{E}(i) = \text{Expected number of rounds to first visit } i - 1 \text{ wealth} + \mathbb{E}(i - 1)$$

Base Case : $\mathbb{E}_t = 0$

The expected number of rounds to first visit $i - 1$ wealth can be calculated as by that time the maximum allowed state is a constant i.e $i + W$

Consider the following markov chain:



Let the expected number of steps to first visit state $i - 1$ if starting with initial wealth x be $\mathbb{E}(x)$, then we can obtain a recursive equation as follows:

$$\mathbb{E}(x) = p\mathbb{E}(x + 1) + q\mathbb{E}(x - 1) + 1$$

Base Case: $\mathbb{E}(i - 1) = 0$

Reflective Boundary: At $i + W$, the maximum state allowed, the wealth cannot increase further. Hence, $\mathbb{E}(i + W) = \mathbb{E}(i + W - 1) + 1$ since we can only move to lower states from here.

First equation:

$$\mathbb{E}(i + W) = \mathbb{E}(i + W - 1) + 1$$

Second equation:

$$\mathbb{E}(i + W - 1) = p\mathbb{E}(i + W) + q\mathbb{E}(i + W - 2) + 1$$

$$\mathbb{E}(i + W - 1) = p(\mathbb{E}(i + W - 1) + 1) + q\mathbb{E}(i + W - 2) + 1$$

$$\mathbb{E}(i + W - 1) = \frac{q\mathbb{E}(i + W - 2) + 1 + p}{1 - p}$$

Third equation:

$$\mathbb{E}(i + W - 2) = p\mathbb{E}(i + W - 1) + q\mathbb{E}(i + W - 3) + 1$$

$$\mathbb{E}(i + W - 2) \left(1 - \frac{pq}{1 - p} \right) = \left((1 + p) \frac{p}{1 - p} + 1 \right) + q\mathbb{E}(i + W - 3)$$

$$\mathbb{E}(i + W - 2) = \frac{\left((1 + p) \frac{p}{1 - p} + 1 \right) + q\mathbb{E}(i + W - 3)}{1 - p}$$

Fourth equation:

$$\mathbb{E}(i + W - 3) = p\mathbb{E}(i + W - 2) + q\mathbb{E}(i + W - 4) + 1$$

$$\mathbb{E}(i + W - 3) \left(1 - \frac{pq}{1-p} \right) = \left(\left((1+p) \frac{p}{1-p} + 1 \right) \frac{p}{1-p} + 1 \right) + q\mathbb{E}(i + W - 4)$$

$$\mathbb{E}(i + W - 3) = \frac{\left(\left((1+p) \frac{p}{1-p} + 1 \right) \frac{p}{1-p} + 1 \right) + q\mathbb{E}(i + W - 4)}{1-p}$$

In general we can conclude that:

$$\mathbb{E}(i + W - x) = \frac{\left(\left(\left((1+p) \frac{p}{1-p} + 1 \right) \frac{p}{1-p} + 1 \right) \dots \text{"}x-1\text{" iterations} \right) + q\mathbb{E}(i + W - x - 1)}{1-p}$$

$$\mathbb{E}(y) = \frac{\left(\left(\left((1+p) \frac{p}{1-p} + 1 \right) \frac{p}{1-p} + 1 \right) \dots \text{"}i + W - y - 1\text{" iterations} \right) + q\mathbb{E}(y - 1)}{1-p}$$

Putting $y = i$, we get:

$$\mathbb{E}(i) = \frac{\left(\left(\left((1+p) \frac{p}{1-p} + 1 \right) \frac{p}{1-p} + 1 \right) \dots \text{"}W-1\text{" iterations} \right)}{(1-p)}$$

This is some constant X independent of i , which can be easily evaluated by a simple $W-1$ iterations loop.

Now, coming back to our original recurrence equation:

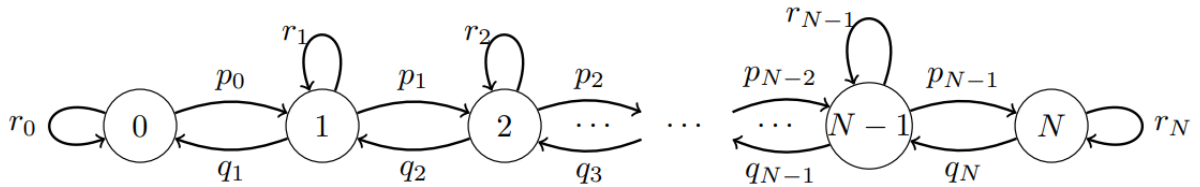
$$\mathbb{E}(i) = X + \mathbb{E}(i-1)$$

Thus

$$\mathbb{E}(k) = X \times (k - t) + \mathbb{E}(t) = X \times (k - t)$$

Question 4

Consider the variations in the price of a particular stock. This can be modeled as a markov chain with a finite state space $\{0, 1, 2, \dots, N\}$. At the end of each time step, the price of the stock can increase or decrease by at most 1.



The transition probabilities can be specified by $\{(p_k, r_k, q_k)\}_{k=0}^N$, where $p_k + r_k + q_k = 1$ for each k and

- p_k is the probability of moving from k to $k+1$ when $0 \leq k < n$,
- q_k is the probability of moving from k to $k-1$ when $0 < k \leq n$,
- r_k is the probability of remaining at k when $0 \leq k \leq n$,
- $q_0 = p_n = 0$.

a) What is the stationary distribution π across the prices of the stock? What is the expected price of the stock in the steady state?

Solution:

Let $\pi = (\pi_0, \pi_1, \dots, \pi_N)$

In stationary distribution: $\pi = \pi P$ where P is the Transition Probability Matrix.

$$P = \begin{bmatrix} r_0 & p_0 & \cdots & 0 & 0 & 0 \\ q_1 & r_1 & p_1 & \cdots & 0 & 0 \\ 0 & q_2 & r_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & q_{N-1} & r_{N-1} & p_{N-1} \\ 0 & 0 & 0 & 0 & q_N & r_N \end{bmatrix}$$

$$(\pi_0, \pi_1, \dots, \pi_N) = (\pi_0, \pi_1, \dots, \pi_N) \begin{bmatrix} r_0 & p_0 & \cdots & 0 & 0 & 0 \\ q_1 & r_1 & p_1 & \cdots & 0 & 0 \\ 0 & q_2 & r_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & q_{N-1} & r_{N-1} & p_{N-1} \\ 0 & 0 & 0 & 0 & q_N & r_N \end{bmatrix}$$

Equation 1:

$$\pi_0 = \pi_0 r_0 + \pi_1 q_1 = \pi_0(1 - p_0) + \pi_1 q_1$$

$$\pi_1 = \pi_0 \frac{p_0}{q_1}$$

Equation 2:

$$\pi_1 = \pi_0 p_0 + \pi_1 r_1 + \pi_2 q_2 = \pi_0 p_0 + \pi_1(1 - p_1 - q_1) + \pi_2 q_2$$

$$0 = \pi_1 q_1 - \pi_1(p_1 + q_1) + \pi_2 q_2$$

$$\pi_2 = \pi_1 \frac{p_1}{q_2}$$

This continues for every π_i and we can say that for $0 < i \leq N$:

$$\pi_i = \pi_{i-1} \frac{p_{i-1}}{q_i} = \pi_0 \frac{\prod_{j=0}^{i-1} p_j}{\prod_{j=1}^i q_j}$$

$$\sum_{i=0}^N \pi_i = 1$$

Using the above two equations we can compute the **stationary distribution** π . **Expected Price** can be calculated once π has been obtained as follows:

$$\mathbb{E}_p = \sum_{i=0}^N i \pi_i$$

b) What is the expected amount of time it takes for the price of the stock to reach b given that it's initial price is a ? ($0 \leq a \leq b \leq N$)

Solution:

Let $\mathbb{E}_x$ represent the expected time to go from state x to state b .

We can form a recursive relation as follows:

$$\mathbb{E}_x = p_x \mathbb{E}_{x+1} + r_x \mathbb{E}_x + q_x \mathbb{E}_{x-1} + 1$$

Base Case: $\mathbb{E}_b = 0$

(As going from state b to state b , considering first visit no time is needed)

This Base case can be used to solve the following system of linear equations:

$$\begin{aligned} (1 - r_0) \mathbb{E}_0 - p_0 \mathbb{E}_1 &= 1 \\ (1 - r_1) \mathbb{E}_1 - p_1 \mathbb{E}_2 - q_1 \mathbb{E}_0 &= 1 \\ \vdots & \\ (1 - r_{b-2}) \mathbb{E}_{b-2} - p_{b-2} \mathbb{E}_{b-1} - q_{b-2} \mathbb{E}_{b-3} &= 1 \\ (1 - r_{b-1}) \mathbb{E}_{b-1} - p_{b-1} \mathbb{E}_b - q_{b-1} \mathbb{E}_{b-2} &= 1 \\ \cancel{(1 - r_b) \mathbb{E}_b - p_b \mathbb{E}_{b+1} - q_b \mathbb{E}_{b-1} &= 1} \end{aligned}$$

The above Equation doesn't hold as if we start at the state b , then there is no going back or front, instead use the base case Equation:

$$\begin{aligned} \mathbb{E}_b &= 0 \\ (1 - r_{b+1}) \mathbb{E}_{b+1} - p_{b+1} \mathbb{E}_{b+2} - q_{b+1} \mathbb{E}_b &= 1 \\ \vdots & \\ (1 - r_{N-1}) \mathbb{E}_{N-1} - p_{N-1} \mathbb{E}_N - q_{N-1} \mathbb{E}_{N-2} &= 1 \\ (1 - r_N) \mathbb{E}_N - q_N \mathbb{E}_{N-1} &= 1 \end{aligned}$$

$$A = \begin{bmatrix} 1 - r_0 & -p_0 & 0 & 0 & \cdots & 0 \\ -q_1 & 1 - r_1 & -p_1 & 0 & \cdots & 0 \\ 0 & -q_2 & 1 - r_2 & -p_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & -p_{N-1} \\ 0 & 0 & 0 & 0 & \cdots & 1 - r_N \end{bmatrix}$$

Note: The matrix A is not the usual tri-diagonal matrix, in the b^{th} row we have all 0's except $A[b][b]$, which is 1. The RHS matrix also has all 1's except one 0 at $\text{RHS}[b]$ i.e in the b^{th} row.

$$A \begin{bmatrix} \mathbb{E}_0 \\ \mathbb{E}_1 \\ \vdots \\ \vdots \\ \mathbb{E}_{N-1} \\ \mathbb{E}_N \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 0 \\ \vdots \\ 1 \\ 1 \end{bmatrix}$$

This matrix equation is solved using **numpy**.

Codes

All implementations are written in Python. Each subsection below contains the complete code for the corresponding question.

Question 1 (ques1.py)

```
1 def win_probability(p, q, k, N):
2     """
3     Return the probability of winning a game of chance.
4
5     p : float, 0 < p < 1, probability of winning a round
6     q : float, q = 1 - p, probability of losing a round
7     k : int, starting wealth
8     N : int, maximum wealth
9     """
10    if (N == 0):
11        return 0
12    elif (N == 1):
13        return k
14    r = q / p
15    if p != q:
16        return (1 - r**k) / (1 - r**N)
17    return k / N
18
19
20 def limit_win_probability(p, q, k):
21     """
22     Return the probability of winning when the maximum wealth is infinity.
23
24     p : float, 0 < p < 1, probability of winning a round
25     q : float, q = 1 - p, probability of losing a round
26     k : int, starting wealth
27     """
28    if q < p:
29        return 1 - (q / p)**k
30    return 0
31
32
33 def game_duration(p, q, k, N):
34     """
35     Return the expected number of rounds to either win or get ruined.
36
37     p : float, 0 < p < 1, probability of winning a round
38     q : float, q = 1 - p, probability of losing a round
39     k : int, starting wealth
40     """
41    r = q / p
42    if p != q:
43        return k / (q - p) + N / (p - q) * (1 - r**k) / (1 - r**N)
44    return -k**2 + k * N
```

Question 2 (ques2.py)

```
1 import math
2
3 def decimalToBinary(num, k_prec):
4     """
5     Convert the decimal fraction `num` to its binary representation
6     using `k_prec` bits after the decimal point. The expansion of
7     num is assumed to terminate before k_prec bits.
8     """
9     binary = ""
10    Integral = int(num)
11    fractional = num - Integral
12
13    while (Integral):
14        rem = Integral % 2
15        binary += str(rem)
16        Integral //= 2
17
18    binary = binary[::-1] # Reverse the string
19    binary += '.'
20
21    while k_prec:
22        fractional *= 2
23        fract_bit = int(fractional)
24        if fract_bit == 1:
25            fractional -= fract_bit
26            binary += '1'
27        else:
28            binary += '0'
29        k_prec -= 1
30
31    return binary
32
33
34 def win_probability(p, q, k, N):
35     """
36     Return the probability of winning while gambling aggressively.
37
38     p : float,  $0 < p < 1$ , probability of winning a round
39     q : float,  $q = 1 - p$ , probability of losing a round
40     k : int, starting wealth
41     N : int, maximum wealth
42     """
43     if k == 0:
44         return 0
45     elif k == N:
46         return 1
47     k_prec = math.ceil(math.log(N, 2)) + 100
48     s = decimalToBinary(k / N, k_prec)
49     while s[-1] == '0':
50         s = s[:-1]
51     s = s.split('.')
52     s = s[1]
53
54     def dp(s, i, p, q):
55         if i == len(s):
56             return 0
57         if s[i] == '1':
```

```

58         return q * dp(s, i + 1, p, q) + p
59     else:
60         return p * dp(s, i + 1, p, q)
61
62     return dp(s, 0, p, q)
63
64
65 def game_duration(p, q, k, N, cache=None):
66     """
67     Return the expected number of rounds to either win or get ruined
68     while gambling aggressively.
69
70     p : float, 0 < p < 1, probability of winning a round
71     q : float, q = 1 - p, probability of losing a round
72     k : int, starting wealth
73     N : int, maximum wealth
74     """
75     if k == 0:
76         return 0
77     elif k == N:
78         return 0
79     k_prec = math.ceil(math.log(N, 2)) + 100
80     s = decimalToBinary(k / N, k_prec)
81     while s[-1] == '0':
82         s = s[:-1]
83     s = s.split('.')
84     s = s[1]
85
86     def dp(s, i, p, q):
87         if i == len(s):
88             return 0
89         if s[i] == '1':
90             return 1 + q * dp(s, i + 1, p, q)
91         else:
92             return 1 + p * dp(s, i + 1, p, q)
93
94     return dp(s, 0, p, q)

```

Question 3 (ques3.py)

```

1 def func(x, p):
2     return x * p / (1 - p) + 1
3
4
5 def game_duration(p, q, k, t, W):
6     """
7     Return the expected number of rounds the gambler will play before quitting.
8
9     p : float, 0 < p < 1, probability of winning a round
10    q : float, q = 1 - p, probability of losing a round
11    k : int, starting wealth
12    t : int, t < k, the gambler will quit if she reaches t
13    W : int, the threshold on maximum wealth the gambler can reach
14    """
15    x = p + 1

```

```

16     for i in range(W - 1):
17         x = func(x, p)
18     return x * (k - t) / (1 - p)

```

Question 4 (ques4.py)

```

1  import numpy as np
2
3  def stationary_distribution(p, q, r, N):
4      """
5      Return a list of size N+1 containing the stationary distribution
6      of the Markov chain.
7
8      p : array of size N+1, 0 < p[i] < 1, probability of price increase
9      q : array of size N+1, 0 < q[i] < 1, probability of price decrease
10     r : array of size N+1, r[i] = 1 - p[i] - q[i], probability of no change
11     N : int, the maximum price of the stock
12     """
13     arr = [0] * (N + 1)
14     arr[0] = 1
15     for i in range(1, N + 1):
16         arr[i] = arr[i - 1] * (p[i - 1] / q[i])
17     # arr looks like:
18     # [1, p[0]/q[1], p[0]*p[1]/q[1]q[2], p[0]*p[1]*p[2]/q[1]q[2]q[3], ...]
19     # Normalize the array
20     total = 0
21     for i in range(N + 1):
22         total += arr[i]
23     normalize = 1 / total
24     for i in range(N + 1):
25         arr[i] *= normalize
26     return arr
27
28
29  def expected_wealth(p, q, r, N):
30      """
31      Return the expected wealth of the gambler in the long run.
32
33      p : array of size N+1, probability of price increase
34      q : array of size N+1, probability of price decrease
35      r : array of size N+1, probability of price remaining the same
36      N : int, the maximum price of the stock
37      """
38     pi = stationary_distribution(p, q, r, N)
39     total = 0
40     for i in range(N + 1):
41         total += i * pi[i]
42     return total
43
44
45  def expected_time(p, q, r, N, a, b):
46      """
47      Returns the expected time to reach state b from state a.
48
49      p : array of size N+1, probability of price increase

```



```
50     q : array of size N+1, probability of price decrease
51     r : array of size N+1, probability of price remaining the same
52     N : int, the maximum price of the stock
53     a : int, the starting price
54     b : int, the target price
55     """
56     if a == b:
57         return 0.0
58
59     # Create the coefficient matrix A of size (N+1)x(N+1)
60     A = np.zeros((N + 1, N + 1))
61     rhs = np.ones(N + 1)      # Right-hand side for the equations
62     rhs[b] = 0                # Expected time to reach target state b is 0
63     # A(N+1, N+1) E(N+1, 1) = rhs(N+1, 1)
64     for i in range(N + 1):
65         A[i, i] = 1 - r[i]      # Main diagonal
66         if i > 0:
67             A[i - 1, i] = -p[i - 1] # Lower diagonal
68         if i <= N - 1:
69             A[i + 1, i] = -q[i + 1] # Upper diagonal
70     for i in range(N + 1):
71         if i == b:
72             A[b][i] = 1
73         else:
74             A[b][i] = 0
75
76     # Solve for E
77     E = np.linalg.solve(A, rhs)
78     return E[a] # Expected time from state a to reach target state b
```